COURS – PHP ORIENTÉ OBJET + MINI-PROJET MVC

Par Robert DIASSÉ

(Préparation obligatoire avant Laravel)

1. Introduction générale

Ce module a un objectif simple : **Vous donner les bases solides en POO et en architecture MVC** afin que vous soyez opérationnels pour comprendre Laravel.

Laravel **n'est pas magique**.
Il repose sur :

- des **classes** (modèles, contrôleurs, middlewares…)
- des **objets** (req, res, user…)
- l'**autoloading** (Composer)
- l'**organisation MVC**
- les **routes**
- les **requêtes HTTP**
- PDO → ensuite Eloquent ORM

Ce module vous faire **reconstruire manuellement** ce que Laravel automatise.

---

2. Rappel fondamental : qu'est-ce qu'un programme orienté objet ?

La POO repose sur quatre notions essentielles :

1) **Classes**

Plan de construction d'un objet.
Métaphore : un plan d'une maison → permet de construire plusieurs maisons.

2) **Objets**

Instances concrètes de la classe.
Métaphore : les maisons réellement construites.

3) **Attributs (propriétés)**

Les données appartenant à l'objet.
Métaphore : couleur, nombre de pièces.

4) **Méthodes**

Les actions de l'objet.
Métaphore : ouvrir une porte, allumer la lumière.

---

3. Exemple simple d'une classe

```
class Person {
    public $name;

    public function speak() {
        return "Bonjour, je m'appelle " . $this->name;
    }
}

$p = new Person();
$p->name = "Awa";
echo $p->speak();
```

Explication simple

- `class` = modèle
- `new Person()` = création d'un objet
- `$this->name` = accéder à un attribut interne
- `speak()` = action de l'objet

---

4. Le **constructeur**

Permet d'initialiser un objet automatiquement.

```
class User {
    private $email;

    public function __construct($email)
    {
        $this->email = $email;
```

```
    }
}
```

---

5. Encapsulation : public / private / getters

```
class BankAccount {
    private $solde = 0;

    public function deposit($amount) {
        $this->solde += $amount;
    }

    public function getSolde() {
        return $this->solde;
    }
}
```

---

6. Héritage

```
class Animal {
    public function eat(){
        return "Je mange.";
    }
}

class Cat extends Animal {
    public function meow() {
        return "Miaou !";
    }
}
```

---

7. Interfaces (contrats)

```
interface Payment {
    public function pay($amount);
}

class OrangeMoney implements Payment {
    public function pay($amount) {
        return "Paiement OM de $amount";
```

```
    }
}
```

---

8. Notion de **composition**

Laravel l'utilise énormément ("un contrôleur utilise un modèle", etc.).

```
class Engine {}

class Car {
    private $engine;

    public function __construct() {
        $this->engine = new Engine();
    }
}
```

---

9. Autoloading : indispensable avant Laravel

Laravel utilise **Composer autoload**.
Nous allons créer **un autoloader manuel**.

Exemple

```
spl_autoload_register(function ($class) {
    require_once __DIR__ . "/../app/$class.php";
});
```

---

10. Architecture MVC – Préparation essentielle pour Laravel

```
project/
│
├── public/
│    └── index.php      # point d'entrée
│
├── app/
│   ├── Controllers/
│   ├── Models/
```

```
│   └── Core/           # autoloader + Database
│
└── views/
    └── services/
```

Pourquoi MVC ?

- **Model** = accéder aux données (comme Eloquent)
- **View** = HTML (comme Blade)
- **Controller** = logique (comme dans Laravel)

---

11. Workflow complet du projet (schéma explicatif)

Utilisateur → Navigue sur /services

1. La requête arrive dans `public/index.php`
2. Le routeur lit l'URL → appelle `ServiceController@index`
3. Le contrôleur appelle `Service::all()`
4. Le modèle interroge la DB via `PDO`
5. La vue affiche les résultats

Schéma (texte)

```
Navigateur → Route → Controller → Model → Database
                                    ↓
                                Retour données
                            Controller → View → HTML
```

C'est **exactement** ce que Laravel fait.

---

12. Mise en place du projet (cours pratique)

✔ 1. Créer la structure

```
project/
  public/
    index.php
  app/
    Controllers/
    Models/
    Core/
  views/
    services/
```

✔ 2. Création du **routeur**

`public/index.php`

```php
<?php
require "../app/Core/autoload.php";

$uri = $_SERVER['REQUEST_URI'];

if ($uri === "/" || $uri === "/services") {
    $controller = new ServiceController();
    $controller->index();
}

if ($uri === "/create") {
    $controller = new ServiceController();
    $controller->create();
}

if ($uri === "/store" && $_SERVER['REQUEST_METHOD'] === "POST") {
    $controller = new ServiceController();
    $controller->store();
}
```

✔ 3. L'autoloader

`app/Core/autoload.php`

```php
<?php

spl_autoload_register(function($class){
    $class = str_replace("\\", "/", $class);
    require __DIR__ . "/../" . $class . ".php";
});
```

✔ 4. Connexion à la base de données

`app/Core/Database.php`

```php
<?php

class Database {
    private static $pdo;
```

```
    public static function connect() {
        if (!self::$pdo) {
            self::$pdo = new PDO("mysql:host=localhost;dbname=mini", "root", "");
            self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        }
        return self::$pdo;
    }
}
```

✔ 5. Le modèle Service (équivalent Eloquent)

`app/Models/Service.php`

```
<?php

class Service {
    public $id;
    public $title;
    public $description;
    public $price;

    public static function all() {
        $db = Database::connect();
        return $db->query("SELECT * FROM services")
                  ->fetchAll(PDO::FETCH_CLASS, Service::class);
    }

    public function save() {
        $db = Database::connect();
        $stmt = $db->prepare("INSERT INTO services (title, description, price)
VALUES (?,?,?)");
        return $stmt->execute([$this->title, $this->description, $this->price]);
    }
}
```

✔ 6. Le contrôleur

`app/Controllers/ServiceController.php`

```
<?php

class ServiceController {

    public function index() {
        $services = Service::all();
        require "../views/services/index.php";
```

```
    }

    public function create() {
        require "../views/services/create.php";
    }

    public function store() {
        $service = new Service();
        $service->title = $_POST['title'];
        $service->description = $_POST['description'];
        $service->price = $_POST['price'];
        $service->save();
        header("Location: /");
    }
}
```

---

13. Les vues

Liste des services

`views/services/index.php`

```
<h1>Liste des services</h1>

<a href="/create">Nouveau service</a>

<table border="1">
    <tr>
        <th>Titre</th>
        <th>Description</th>
        <th>Prix</th>
    </tr>

    <?php foreach($services as $s): ?>
    <tr>
        <td><?= $s->title ?></td>
        <td><?= $s->description ?></td>
        <td><?= $s->price ?> F</td>
    </tr>
    <?php endforeach; ?>
</table>
```

Formulaire d'ajout

`views/services/create.php`

```
<h1>Ajouter un service</h1>

<form method="POST" action="/store">
    <input type="text" name="title" placeholder="Titre" required><br><br>
    <textarea name="description" placeholder="Description"></textarea><br><br>
    <input type="number" name="price" placeholder="Prix" required><br><br>
    <button type="submit">Enregistrer</button>
</form>
```

---

14. Exercices pratiques pour les étudiants

1. Ajouter l'édition (`/edit`, `/update`)
2. Ajouter la suppression (`/delete`)
3. Ajouter les catégories + relation services/catégories
4. Ajouter un système de validation (classe Validator)
5. Ajouter un système de session + flash messages

---

15. Ce que vous comprennez après ce projet

À la fin, vous devez comprendre:

- Pourquoi Laravel a des contrôleurs
- Pourquoi Laravel a des modèles
- Pourquoi il existe Blade
- Comment fonctionnent les routes
- Comment fonctionne Eloquent
- Pourquoi Composer autoload est essentiel
- Pourquoi l'architecture MVC simplifie tout

Ce projet sert de **transition parfaite** vers Laravel.